

FeedFlow UX Framework: Specifications

Context and Purpose

The web is shifting from desktop-centric to mobile-first consumption. Research shows that most users hold their phones vertically and interact with content in a feed-like way ¹. Apps like TikTok have normalized vertical infinite scroll and short video formats, and these patterns are beginning to influence websites ². While vertical storytelling is popular on social media, there is currently no established standard for implementing feed-like, narrative experiences on mobile websites. Many guidelines caution against using vertical video on websites due to design challenges ³. Existing scroll patterns, such as CSS scroll snap, provide building blocks for controlled full-page section navigation ⁴, but a unified UX framework is missing.

FeedFlow aims to fill this gap by providing a design and development framework for creating narrative, vertical-scrolling mobile websites inspired by social feed behaviours. It combines design thinking, UX best practices, accessibility, and modern web technologies.

Key Principles

1. **Mobile-first narrative flow** – The mobile experience is treated as a distinct product, not just a responsive adaptation. Users consume content sequentially, so each section acts as a "slide" in a vertical story.
2. **Full-viewport sections with scroll snap** – Each segment occupies ~100vh and uses CSS `scroll-snap-type: y mandatory` to align sections precisely within the viewport ⁴. This provides feed-like "swiping" from one screen to the next.
3. **Sequential storytelling** – Content is broken into digestible fragments. Phrases, images, and micro-interactions guide the user through a narrative arc (intrigue → problem → solution → examples → credibility → CTA).
4. **Micro interactions and motion** – Subtle animations (fade in, slide up, parallax) reinforce the narrative and reward scrolling. Use frameworks like Framer Motion or GSAP with care and respect `prefers-reduced-motion`.
5. **Strong typography and minimal copy** – Large headlines and short lines of text ensure readability on small screens. Avoid long paragraphs.
6. **Ethical engagement** – Borrow the engagement mechanics of feeds without creating addictive patterns. Avoid manipulative infinite scroll; provide clear endpoints and CTAs.
7. **Accessibility and performance** – Follow WCAG guidelines: ensure keyboard navigation, support screen readers, and provide reduced-motion modes ⁵. Optimize images with `next/image` and minimize blocking scripts.

Implementation Overview

1. Assess Existing Content

- Catalogue all sections and components of the current desktop site.
- Identify key messages and assets (copy, images, videos) that must be preserved.
- Determine if any content must be restructured or condensed for mobile narrative flow.

2. Prepare the Technical Stack

- **Base framework:** Use Next.js (already in the project).
- **Animation & scroll utilities:** Check existing dependencies. If present, reuse:
 - `framer-motion` for element animations.
 - `gsap` and `ScrollTrigger` for scroll-synchronized effects.
 - `@studio-freight/lenis` for smooth scrolling.
- **Optional libraries to install** (only if missing and needed):
 - `framer-motion` for animations.
 - `gsap` + `ScrollTrigger` for advanced scroll control.
 - `lenis` for easing scroll and inertial feel.
- Avoid heavy 3D libraries unless essential.

Installation example (using npm):

```
npm install framer-motion @studio-freight/lenis gsap
```

3. Create Core Layout Components

Implement reusable components to structure the FeedFlow experience:

- `MobileFullSection`: A wrapper that applies `height: 100vh`, `scroll-snap-align: start`, and optional background styling.
- `MobileScrollContainer`: The parent wrapper with `overflow-y: scroll` and `scroll-snap-type: y mandatory`. For smoother motion, wrap with `lenis` provider.
- `MobileStorySection`: A component that accepts props for text, media, and animation options. Use variant controls to alternate layouts.
- `MobileStickyCTA`: A bottom or end-of-flow call-to-action that appears after key narrative steps.

Define breakpoints in CSS or Tailwind to enable this layout only on small screens (`max-width: md`). Preserve existing desktop layout beyond that breakpoint.

4. Build the Mobile Narrative

Transform each section from the desktop site into FeedFlow sections:

1. **Intrigue block** – Present a bold statement or question to hook the user. Use large typography with minimal text.

2. **Problem block** – Briefly articulate the pain or gap that the service addresses.
3. **Solution block** – Explain the unique value proposition. Keep copy concise.
4. **Case visuals block** – Showcase example projects or results. Use horizontal scroll within the vertical flow if needed.
5. **Process block** – Visualize the service steps as a vertical timeline with clear steps and subtle animations.
6. **Credibility block** – Highlight testimonials or credentials in a minimal format.
7. **CTA block** – Provide a clear invitation to engage (e.g., “¿Tu proyecto parece uno más?” with a button).

Alternate backgrounds and motion to create rhythm and maintain interest. Use `framer-motion` variants to orchestrate fade-ins and slide-ups as sections enter the viewport.

5. Interactivity and Motion

- Wrap the entire scroll container with `Lenis` for smooth inertial scrolling.
- Use CSS `scroll-snap` for primary section snapping ⁴.
- Implement intersection observers or `ScrollTrigger` to trigger animations when sections enter view.
- Respect `prefers-reduced-motion`: disable or simplify animations for users who prefer reduced motion.

Example of a section with motion:

```
import { motion } from 'framer-motion';

export function IntrigueSection() {
  return (
    <section className="h-screen flex items-center justify-center scroll-snap-align-start">
      <motion.h1
        initial={{ opacity: 0, y: 50 }}
        whileInView={{ opacity: 1, y: 0 }}
        transition={{ duration: 0.6 }}
        className="text-4xl font-bold"
      >
        Los proyectos inmobiliarios no se venden. Se perciben.
      </motion.h1>
    </section>
  );
}
```

6. Accessibility and Testing

- Add `tabIndex` and keyboard controls for interactive elements.
- Ensure `aria-labels` are used on CTA buttons and navigation hints.
- Provide skip links if appropriate.
- Test with screen readers and on physical devices (iOS and Android).

- Validate that scrolling doesn't trap focus or cause orientation issues.

7. Repository Structure

A recommended folder layout:

```
/feedflow-ux/  
  README.md  
  /components/  
    MobileFullSection.jsx  
    MobileScrollContainer.jsx  
    MobileStorySection.jsx  
    MobileStickyCTA.jsx  
  /sections/  
    IntrigueSection.jsx  
    ProblemSection.jsx  
    ...  
  /styles/  
    globals.css  
    feedflow.css (scroll snap utilities, custom animations)  
  /lib/  
    useLenis.tsx (hook to integrate Lenis)
```

- Document each component and its props in the README.
- Include sample pages (e.g., `pages/mobile.tsx`) demonstrating how to assemble the components.
- Provide guidelines for customizing colors, fonts, and spacing to match a brand.

8. Contribution Guidelines

- Use pull requests with clear descriptions.
- Follow coding standards (e.g., ESLint + Prettier).
- Test on multiple devices before merging.
- Document any changes to the core interaction patterns.

9. Ethical and Legal Considerations

- Avoid infinite scroll loops that deliberately obscure the end of content.
- Provide users with clear navigation and the ability to return to previous sections.
- Ensure compliance with accessibility laws (WCAG 2.1), privacy policies, and data protection regulations (e.g., GDPR).

10. Future Directions

- Expand the framework to support interactive data visualizations and real-time content.
- Explore integration with CMS platforms to allow content teams to author FeedFlow sections without coding.

- Gather user feedback to refine patterns and update guidelines.

Conclusion

FeedFlow UX Framework offers a principled approach to translating the addictive yet engaging mechanics of social feeds into ethical, narrative-driven mobile web experiences. By combining established patterns like scroll snap ⁴ with thoughtful storytelling and modern front-end tools, teams can craft immersive journeys that respect users and strengthen brand perception.

¹ ³ The Rise of Vertical Storytelling - Atilus

<https://atilus.com/vertical-storytelling/>

² How TikTok's Addicting UX is Subtly Influencing Website and App Design | by Nanzing Johnmark | Bootcamp | Medium

<https://medium.com/design-bootcamp/how-tiktoks-addicting-ux-is-subtly-influencing-website-and-app-design-60c535786f44>

⁴ ⁵ Scrolling Designs: 8 Patterns and When to Use Each (2026) | Lovable

<https://lovable.dev/guides/scrolling-designs-patterns-when-to-use>